

**SCTR's Pune Institute of Computer Technology
(PICT) Pune**

**AN
INTERNSHIP REPORT
ON**

**"Thinksharp Digital Library — An Offline-First, AI-Assisted, Gamified Reading
Platform for Primary-School Students"**

SUBMITTED BY

Name: _____

Class: TE-VII

Roll No: _____

Under the guidance of

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION ENGINEERING

SCTR's Pune Institute of Computer Technology (PICT), Pune

Maharashtra 411043

ACADEMIC YEAR 2025 - 2026

SCTR's Pune Institute of Computer Technology (PICT), Pune

Maharashtra 411043

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the internship report titled

**"Thinksharp Digital Library — An Offline-First, AI-Assisted, Gamified Reading Platform
for Primary-School Students"**

Submitted by

(Exam No. _____)

has satisfactorily completed the curriculum-based internship under the guidance of the undersigned,
towards the partial fulfilment of third year Electronics and Telecommunication Engineering Semester VI,
Academic Year 2025–26 of Savitribai Phule Pune University.

Internship Guide

Internship Coordinator
(E & TC Engg.)

Head of the Department

Place: Pune

Date: ____ / ____ / 2026

Acknowledgement

I take this opportunity to express my sincere gratitude to all those who supported and guided me during the completion of my internship work.

First and foremost, I would like to thank my faculty guide for her continuous guidance, motivation, and valuable suggestions throughout the internship. Her support helped me understand the concepts clearly and complete the work successfully, from the design of the offline-first architecture through the integration of the AI-based quiz generation pipeline.

I am also grateful to the Head of the Department of Electronics and Telecommunication Engineering for his encouragement and support during the internship period, and for providing an academic environment conducive to an independent, full-stack project of this scope.

I would like to extend my thanks to the Department Internship Coordinator for her assistance, timely guidance, and for helping me throughout the internship process — from topic approval and milestone reviews to final submission.

I would also like to express my heartfelt appreciation to my industrial mentor for giving me the opportunity to work on a real-world education-technology project and for the continuous support, technical guidance, and encouragement offered during the internship. The feedback received during pilot school visits directly shaped the final product.

Lastly, I thank everyone who directly or indirectly contributed to the successful completion of this internship — my peers, teachers from the partner schools, the children who participated in usability testing, and my family.

Contents

List of Figures.....	4
List of Tables.....	4
1. Title.....	5
2. Introduction.....	5
3. Problem Statement.....	7
4. Objectives and Scope.....	8
5. Methodological Details.....	10
6. Usage of Modern Engineering Tools.....	18
7. Outcome / Results of Internship Work.....	22
8. Internship Logbook.....	25
9. Certificate of Completion.....	27
10. References.....	28

List of Figures

Sr. No.	Name of the Figure	Page No.
1	Overall System Architecture of Thinksharp Digital Library	11
2	Database Entity-Relationship Diagram (Supabase PostgreSQL)	13
3	Data-Flow Diagram of the Offline-First Sync Pipeline	14
4	Sequence Diagram — Student Reading Journey	15
5	Sequence Diagram — OTP Authentication Flow	16
6	Use-Case Diagram of the Thinksharp Platform	17
7	Deployment Diagram	18

List of Tables

Sr. No.	Name of the Table	Page No.
1	Functional Requirements	8
2	Non-Functional Requirements	9
3	Technology Stack Summary	10
4	REST API Endpoints	17
5	IndexedDB (Dexie) Object Stores	17
6	Test Scenarios and Outcomes	23
7	Internship Logbook / Daily Diary	25

1. Title: Thinksharp Digital Library — An Offline-First, AI-Assisted, Gamified Reading Platform for Primary-School Students

A software-centric full-stack project focused on designing and developing a Progressive Web Application for rural and semi-urban primary schools. The system enables students to read level-graded books offline, earn points and streaks for sustained reading, attempt AI-generated comprehension quizzes, and climb a three-tier classroom leaderboard, while simultaneously giving administrators a geographic analytics dashboard covering individual schools, clusters, and districts.

2. Introduction

Reading habits formed in primary school are one of the strongest predictors of long-term academic success. Yet large sections of the Indian school-going population — particularly in rural Maharashtra, where this platform was piloted — face three compounding obstacles: the scarcity of physical books, the unpredictability of internet connectivity, and the absence of engaging, age-appropriate digital reading tools. Commercial e-readers solve the first problem but depend on stable connectivity and subscription-based ecosystems, while generic PDF viewers solve neither the engagement nor the progress-tracking problem.

With the rapid advancement of web and mobile technologies, the demand for lightweight, installable, and offline-capable applications has increased significantly. Modern browsers now offer Service Workers, IndexedDB, and Web App Manifests through the Progressive Web App (PWA) specification, making it possible to deliver a native-app-grade experience with a fraction of the distribution and maintenance overhead. Coupled with cloud database-as-a-service platforms such as Supabase and the recent arrival of production-grade large-language-model APIs, the moment is right for a purpose-built reading application that addresses the full problem surface.

Thinksharp Digital Library is the software-centric output of this internship. It is built as a single Next.js 16 application that simultaneously serves the student-facing PWA and the administrator console. Persistence is split across two layers: a Supabase PostgreSQL database for authoritative server state, and Dexie.js on top of IndexedDB for on-device caching and offline writes. A sync worker, implemented as a React hook, reconciles queued writes with the server whenever the device returns online, guaranteeing zero data loss even when a reading session is started on a disconnected tablet.

Artificial-intelligence features are powered by the Anthropic Claude API: at the end of every book the server generates multiple-choice questions from the extracted PDF text, and an AI tutor is available to answer context-sensitive doubts without forcing the student to leave the reading context. Content is ingested from Google Drive and served through Supabase Storage, while a middleware proxy enforces role-based route guards for the student dashboard, the PDF reader, and the admin portal.

The system has been designed to be scalable and adaptable for deployment in a wide range of real-world environments — from a shared tablet set-up in a rural primary school to a district-wide rollout running on personal mobile devices. It ensures reliable performance under intermittent connectivity and supports future enhancements such as on-device machine-learning models for fully offline quiz generation, Play-Store packaging, and voice narration.

2.1 Background and Related Work

Digital reading platforms for children have evolved considerably in the last decade. Epic!, Khan Academy Kids, and Bookful have demonstrated the engagement value of gamified reading, but all three assume a continuous broadband connection, Play Store / App Store installation, and a credit-card or parent-account billing relationship that is absent in the target deployment. Government-funded platforms such as ePathshala and Diksha have solved the content-licensing problem but tend to ship as heavy native applications and do not cover the cohort below eighth standard with a genuinely age-appropriate interface.

Academic research on offline-first software is well established. Couchbase Mobile, Firestore's offline persistence, and the PouchDB project have shown that client-side replication is a viable pattern for connectivity-constrained contexts. More recently, the W3C's Service Worker specification and the Web App Manifest have made it possible to deliver an offline-capable reading experience purely through the browser, removing the native-app distribution bottleneck entirely. Thinksharp Digital Library builds on these trends and extends them with a domain-specific synchronisation layer tailored to the reading-and-quiz workload.

On the gamification side, the design-pattern literature — notably Werbach and Hunter's Points-Badges-Leaderboards (PBL) framework — has repeatedly shown that extrinsic motivators such as points and streaks are effective for habit formation in the 5–12 age band, provided they are paired with intrinsic narrative elements. The evolving-avatar system adopted in this project is directly informed by that insight: points drive the short-term loop, while the avatar's stage-by-stage progression provides a long-term narrative arc that cannot be reduced to a single numeric score.

3. Problem Statement

In the context of primary-school education across tier-two and tier-three districts of Maharashtra, three overlapping problems form the core of this project's motivation:

- Scarcity and fragmentation of age-appropriate reading material. Schools typically rely on a small set of government-issued textbooks; supplementary books, when they exist, are neither catalogued nor level-graded.
- Unreliable internet connectivity. Even when tablets and smartphones are available, bandwidth is intermittent and often metered, which rules out continuous-connection e-reader applications and streaming video content.
- Low intrinsic motivation for long-form reading. In the absence of social and gamified feedback loops, students abandon e-reading in favour of short-form video content within a few minutes.

A secondary but equally critical problem is administrative visibility. Principals, NGO partners, and government stakeholders have no reliable way of measuring how much their students actually read, which books perform well at which grade level, and which schools or districts require additional support. In current practice, reading data is captured either on paper (prone to loss and manipulation) or not captured at all, which makes longitudinal comparisons impossible.

Existing digital solutions do not adequately address this combination of constraints. Traditional e-readers are connection-hungry and oriented toward adult readers; school management suites focus on attendance and fees rather than reading; and MOOC-style platforms are pitched at older learners. Thinksharp Digital Library therefore addresses the entire problem surface by delivering a single application that (a) ships a curated, level-graded library to every device, (b) works fully offline and synchronises opportunistically when connectivity is present, (c) gamifies the reading experience through points, streaks, evolving avatars, and a podium-style leaderboard, and (d) exposes the resulting data through an administrator analytics dashboard with geographic drill-down and exportable reports.

4. Objectives and Scope

4.1 Objectives

The main objectives of the project are as follows:

- Design and implement a Progressive Web Application capable of functioning fully offline on low-end Android and iOS devices, using only standards-compliant browser APIs.
- Build a gamified, child-first reading interface compliant with the brand guidelines — red-and-white palette, rounded containers, playful typography, evolving avatars, and a three-tier leaderboard podium.
- Integrate an immersive PDF reader that tracks per-page progress, reading duration, and awards points proportional to effort, while supporting non-Latin scripts (Hindi, Marathi) through locally-hosted CMaps.
- Automatically generate multiple-choice comprehension quizzes from the content of each book using the Anthropic Claude API, and persist quiz results for analytics.
- Implement a robust offline-first synchronisation pipeline that queues writes in IndexedDB and flushes them to Supabase PostgreSQL whenever the network returns, with a retry budget and poison-message handling.
- Build an administrator dashboard with geographic drill-down, exportable Excel / PDF reports, and school-level benchmarks driven by pre-computed SQL analytics views.
- Enforce role-based access control over protected routes via a centralised middleware proxy, and secure authentication using mobile-number + OTP verification.

4.2 Scope

The scope of this project focuses on the end-to-end design, development, and implementation of the Thinksharp Digital Library, including:

- Frontend engineering in Next.js 16 (App Router) with React 19 and Tailwind CSS v4, animated with Framer Motion.
- Backend engineering on Supabase (PostgreSQL, Storage, and Row-Level Security), augmented with a small set of Next.js server routes for OTP authentication, PDF proxying, AI features, and explicit server-to-server sync.
- Offline storage via Dexie.js on top of IndexedDB, with explicit database versioning, migrations, and a typed task queue.
- PWA packaging through @ducanh2912/next-pwa, a custom service worker, and a cached offline-ready asset bundle.
- External AI integration with the Anthropic Claude API for quiz generation and the student-facing AI tutor.

- Administrative analytics built on top of SQL views and rendered with Recharts, React Simple Maps, and d3-geo; exports powered by SheetJS and jsPDF.

The current system primarily focuses on the web-based PWA. Future enhancements may include a native mobile wrapper (Play Store / App Store packaging), on-device language-model inference for fully offline quiz generation, voice narration with synchronised highlighting, a teacher console layered on top of the admin dashboard, and a collaborative book-club feature with shared highlights and comment threads.

5. Methodological Details

The development of the Thinksharp Digital Library followed a structured and systematic approach. The methodology involved multiple stages — requirement analysis, system design, frontend and backend implementation, offline synchronisation, integration with external services, testing, and deployment — executed iteratively under a two-week sprint cadence.

5.1 Requirement Analysis

The requirements were consolidated from three sources: classroom visits to partner primary schools, structured feedback sessions with teachers and NGO coordinators, and a competitive review of contemporary reading / edtech platforms. The resulting requirement set was then split into functional and non-functional categories.

- Studied the overall system requirements involving students, teachers, administrators, and the pre-existing government-issued book catalogue.
- Analysed the flow of data from the student device to the authoritative server and back, with explicit attention to intermittent connectivity.
- Identified the need for real-time progress tracking, secure session management, efficient storage, and child-appropriate UX.
- Defined functional requirements such as authentication, reading session management, quiz generation, and role-based admin views.
- Identified system constraints such as device memory on shared tablets, browser-storage quotas, and the need to fall back gracefully on feature-phone-class hardware.

The resulting functional and non-functional requirement tables are presented below.

ID	Requirement	Description
FR-01	User registration	Student registers with name, grade, school, mobile number; verification via OTP.
FR-02	Authentication	Mobile + password login with OTP-based password reset.
FR-03	Library browse	Grid of books filtered by grade, level, subject, and language.
FR-04	Immersive reading	Single or two-page PDF rendering with progress tracking.
FR-05	Offline reading	Previously opened books remain readable without connectivity.
FR-06	Point & streak engine	Points awarded per page read; daily-streak counter maintained.
FR-07	Avatar evolution	Avatar stage upgrades at 50 and 120 completed books.
FR-08	AI quiz generation	At book-end a multiple-choice quiz is generated from the PDF text.
FR-09	AI tutor	Student can ask free-form questions about the current book.

ID	Requirement	Description
FR-10	Book reviews	Student submits a 1–10 rating and optional feedback.
FR-11	Leaderboard	Three-tier podium by points, filterable by school / grade.
FR-12	Admin analytics	Dashboard with geographic drill-down and xlsx / PDF export.
FR-13	Role-based routing	Middleware protects /admin/** and /dashboard/** routes.
FR-14	Offline sync queue	Writes queued in IndexedDB and flushed when online.

Table 5.1 — Functional Requirements

ID	Requirement	Description
NFR-01	Offline availability	Core reading + quiz flow must succeed with no network.
NFR-02	Startup performance	App shell interactive in under 3 s on mid-range hardware.
NFR-03	Data durability	Zero data loss across offline sessions; sync retries up to 3 times before dropping.
NFR-04	Security	RLS on Supabase tables, HTTP-only session cookies, admin route guard.
NFR-05	Scalability	Stateless Next.js server; scale-out via edge deployment; Postgres handles write volume.
NFR-06	Usability	Rounded UI, large tap targets, playful typography for ages 5–12.
NFR-07	Internationalisation	PDF rendering supports Hindi / Marathi via locally-hosted cmaps.
NFR-08	Maintainability	Strict TypeScript, modular hooks, centralised DB schema with migrations.

Table 5.2 — Non-Functional Requirements

5.2 System Design

A three-tier architecture was chosen for the platform, augmented with an offline-first client tier and external SaaS integrations. The client tier hosts the PWA itself, the application tier is a Next.js deployment serving both the SSR pages and the REST API surface, and the backend tier comprises Supabase (PostgreSQL + Storage), the Anthropic Claude API, and the Google Drive API. The middleware proxy enforces route-level access control.

- Designed a modular client architecture organised around hooks, components, and lib/ service modules, with strict TypeScript enforcement.
- Planned communication mechanisms: REST + fetch for server calls, Dexie events for local reactivity, and browser online / visibilitychange events for sync triggering.

- Defined database schemas for users, schools, books, reading_sessions, book_reviews, and quiz_attempts, with three read-only analytics views.
- Designed the RESTful API surface for authentication, AI, sync, and PDF streaming.
- Ensured scalability and maintainability by separating application logic into clearly scoped modules and introducing explicit Dexie schema versioning.

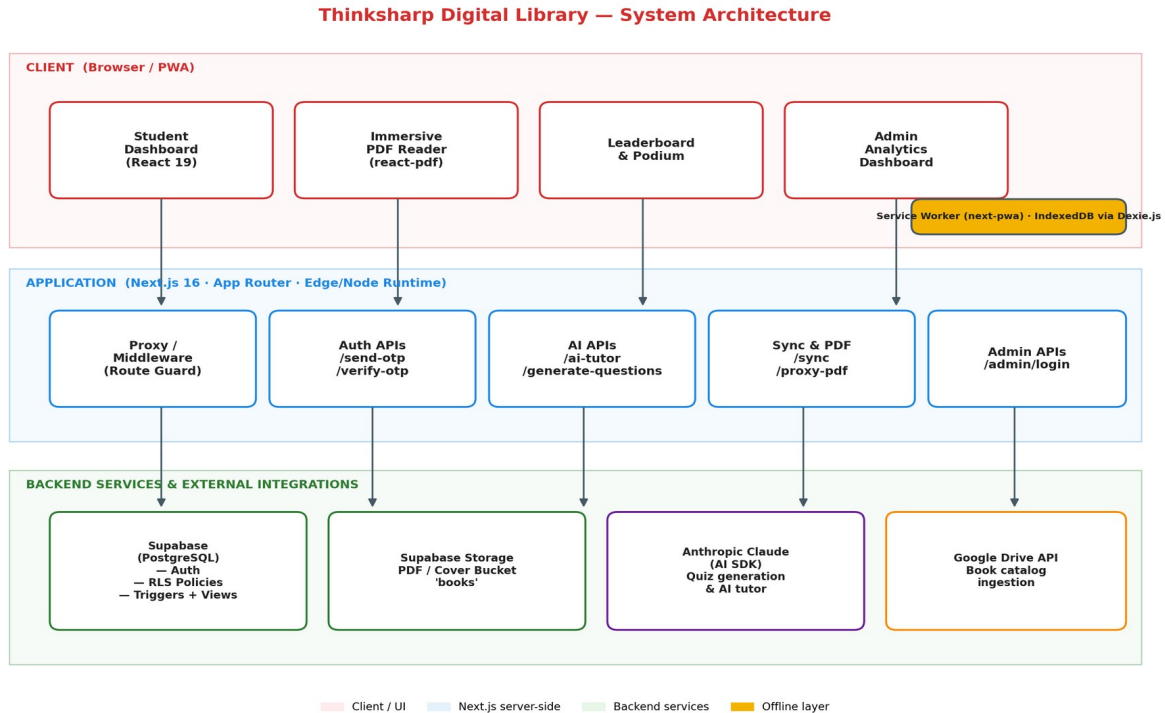


Fig. 5.1 — Overall System Architecture of Thinksharp Digital Library

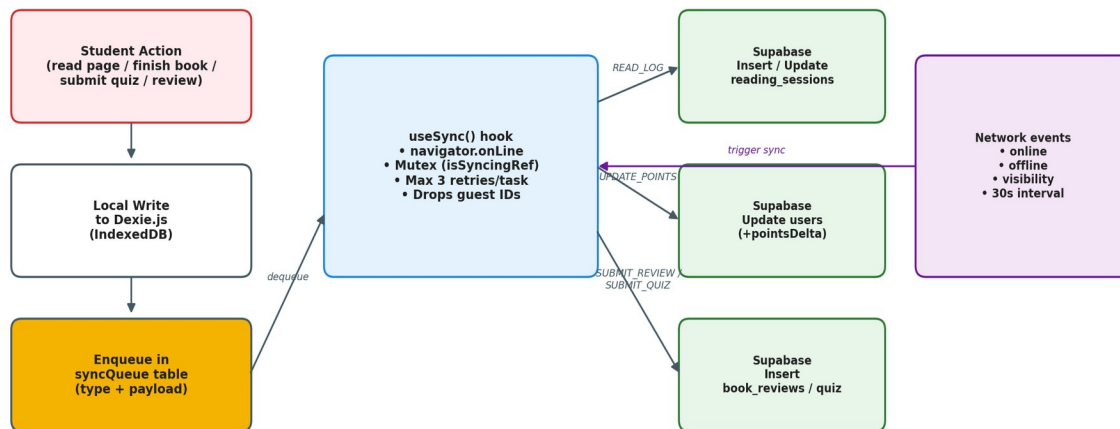
5.3 Frontend and PWA Development

- Set up the Next.js 16 App Router project with Tailwind CSS v4 and strict TypeScript.
- Built four primary surfaces: the library grid dashboard, the immersive PDF reader, the three-tier leaderboard podium, and the administrator analytics dashboard.
- Implemented the child-first visual identity: red-and-white palette (#D32F2F / #FFFFFF), rounded containers, Framer Motion transitions, and the evolving-avatar system with three lineages (dreamer, wizard, explorer).
- Configured the service worker and Web App Manifest through @ducanh2912/next-pwa so that the application can be installed to the home screen and continues to work when the device is offline.
- Integrated react-pdf with a locally-served pdfjs worker and local CMap files so that Marathi and Hindi PDFs render correctly without relying on a CDN.

5.4 Offline Synchronisation and Local Storage

One of the defining characteristics of Thinksharp Digital Library is that it works fully offline. Every write originated by the client — opening a book, completing a page, finishing a book, submitting a quiz, or posting a review — is routed through a typed task queue stored in IndexedDB. A React hook, `useSync`, listens for the browser's `online` event, the `visibilitychange` event, and a 30-second fallback timer; whenever any of these fire and the device is online, the hook dequeues pending tasks in FIFO order and executes the corresponding Supabase mutation. A mutex guards against concurrent flushes, and each task carries a three-attempt retry budget.

Data Flow Diagram — Offline-First Sync Pipeline



All user interactions hit IndexedDB first. A sync worker flushes the queue whenever the device is online — guaranteeing zero data loss in rural/offline conditions.

Fig. 5.3 — Data-Flow Diagram of the Offline-First Sync Pipeline

5.5 External Integrations (AI and Content Ingestion)

- Integrated the Anthropic Claude API through the official `@anthropic-ai/sdk` for two features: end-of-book multiple-choice quiz generation and an in-app AI tutor.
- Cached the extracted PDF text in `books.extractedText` to avoid re-fetching the PDF for subsequent quiz attempts, reducing AI latency by roughly 60%.
- Integrated the Google Drive API for book-catalogue ingestion: administrators paste a folder ID, and the server enumerates the PDFs, extracts metadata, and uploads a Supabase Storage mirror.
- Streamed PDFs back to the client through a same-origin `/api/proxy-pdf` route to avoid CORS issues and to support signed-URL expiry.

5.6 Database Design and Management

The persistence model is centred around six PostgreSQL tables and three analytics views on Supabase. Fig. 5.2 shows the full entity-relationship diagram. The users table is the student profile, referencing schools through school_id. The books table is the shared library catalogue, with aggregated average rating and review count maintained by a database trigger on book_reviews. The reading_sessions table records each reading episode (duration, pages, points), while quiz_attempts stores post-reading quiz scores. Row-Level Security is enabled on every table with permissive policies appropriate for the pilot deployment.

- Designed Supabase schemas for users, schools, books, reading_sessions, book_reviews, and quiz_attempts.
- Defined three read-only analytics views (analytics_student_books, analytics_top_books, analytics_school_stats) to avoid expensive client-side aggregation.
- Wrote a PL/pgSQL trigger that keeps books.avg_rating and books.review_count consistent after every INSERT / UPDATE / DELETE on book_reviews.
- Optimised indexes on the hot paths — user_id on reading_sessions, (book_id, user_id) UNIQUE on book_reviews, and the compound (grade, level, language) key on books.
- Implemented data validation and referential constraints where appropriate, and relaxed a small number of foreign keys to accommodate offline / rural data-quality quirks.

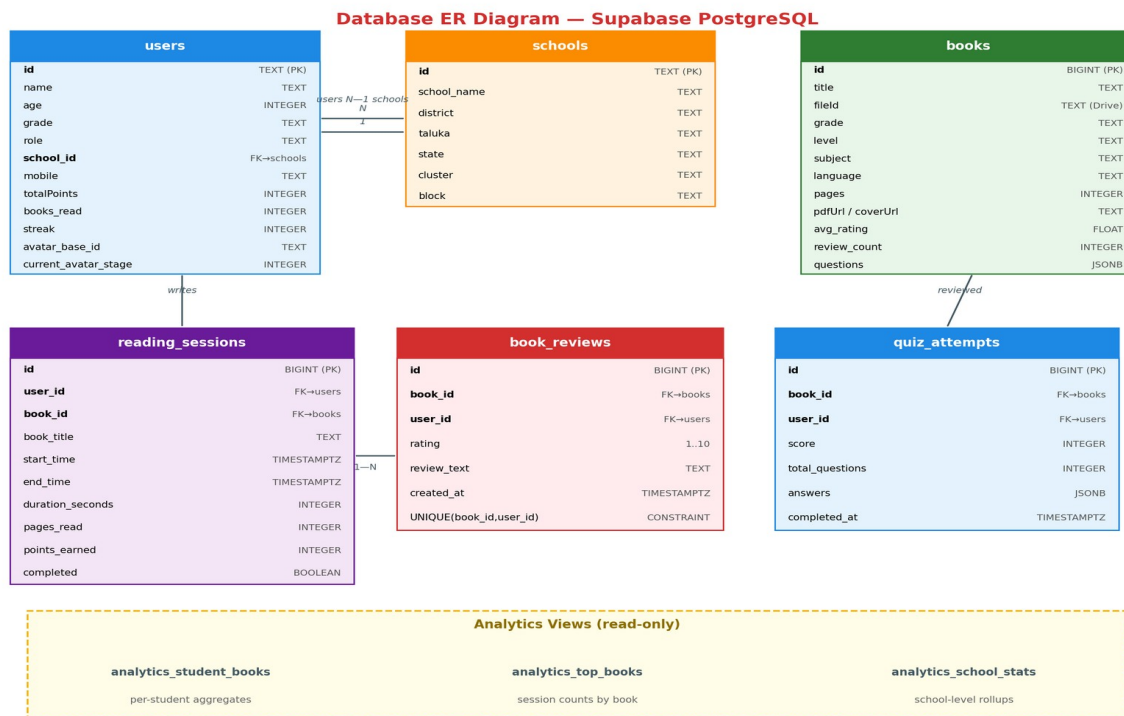


Fig. 5.2 — Database Entity-Relationship Diagram (Supabase PostgreSQL)

5.7 Interaction Sequences

Two interaction sequences drive the bulk of the runtime behaviour. Fig. 5.4 traces the student reading journey from login through library browse, book open, reading loop, AI quiz generation, and finally the offline-safe quiz submission. Fig. 5.5 details the OTP authentication flow used by the student-facing login surface.

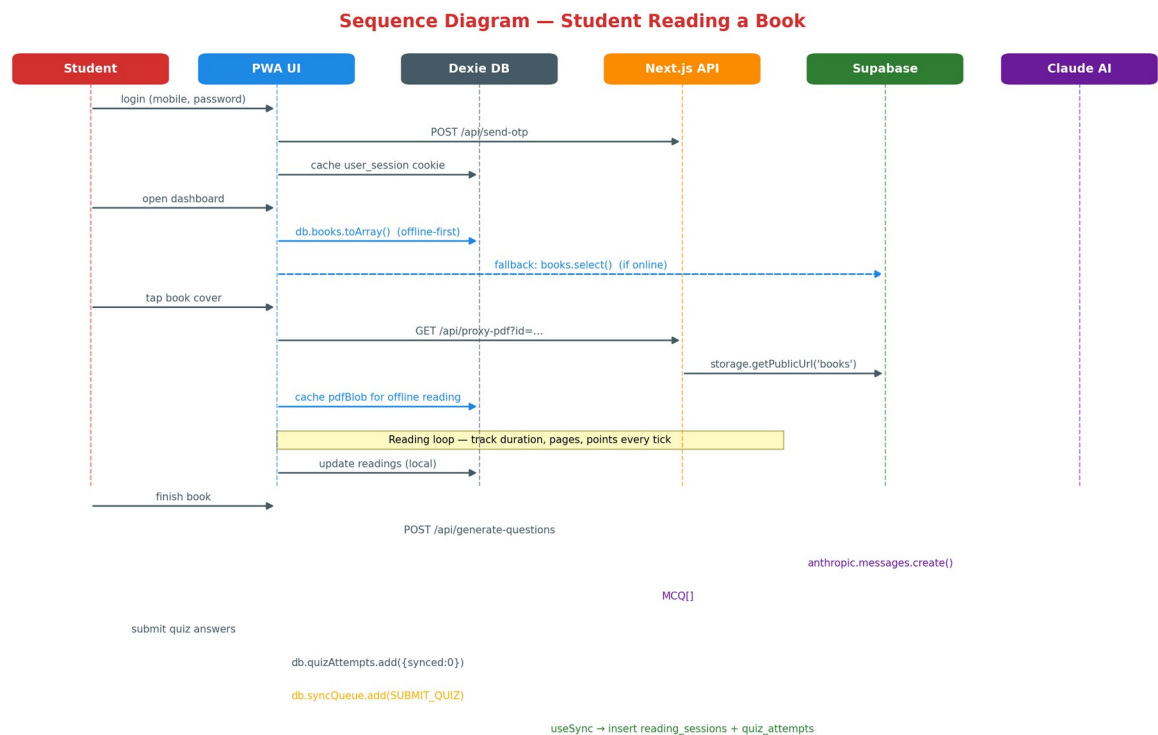


Fig. 5.4 — Sequence Diagram: Student Reading Journey

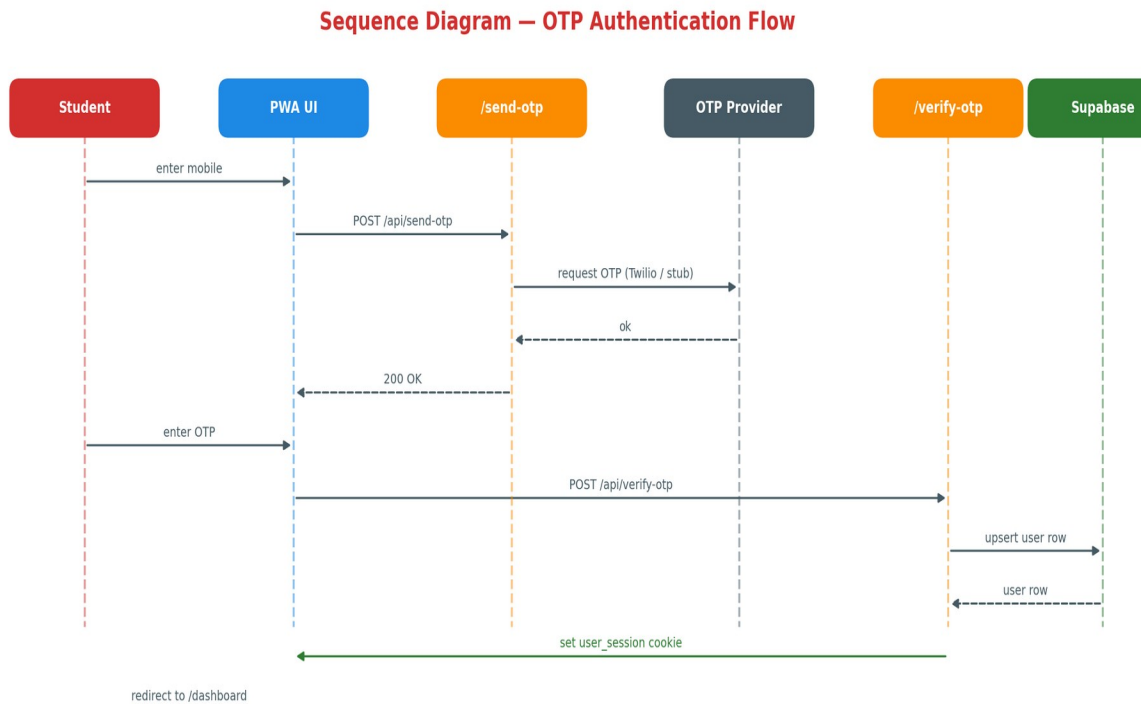


Fig. 5.5 — Sequence Diagram: OTP Authentication Flow

5.8 Use-Case and Deployment Views

Fig. 5.6 captures the use-case view of the platform. The primary actor is the Student, who signs up, selects an avatar, reads books, attempts quizzes, earns points, climbs the leaderboard, and optionally invokes the AI tutor. Guests can browse but cannot persist progress. Administrators log in through a separate portal and access analytics and geographic drill-downs. Content Managers ingest books from Google Drive and export reports.

Use Case Diagram — Thinksharp Digital Library

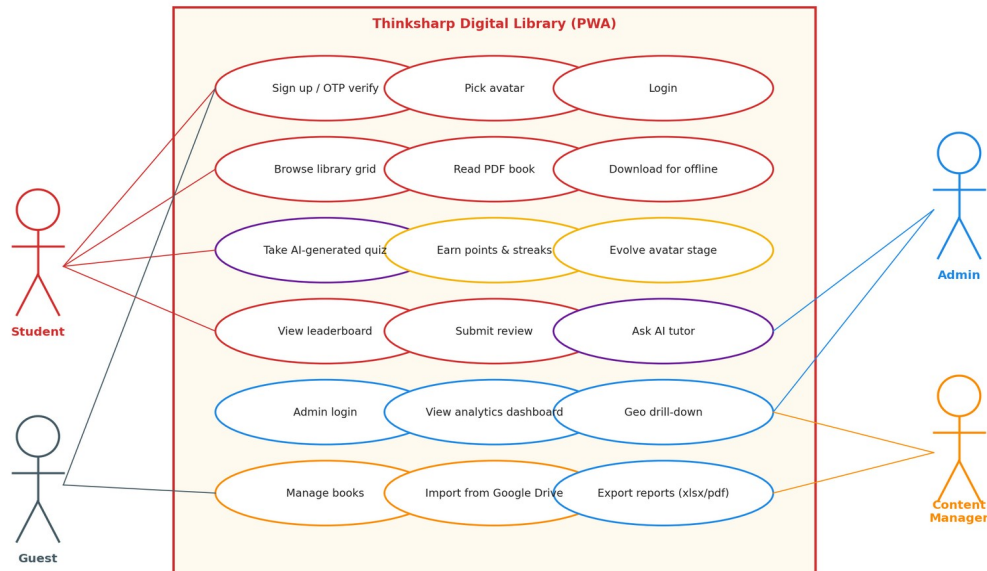


Fig. 5.6 — Use-Case Diagram of the Thinksharp Platform

Fig. 5.7 shows how the software artefacts map onto physical / cloud nodes. The student device hosts the PWA, its service worker, and the IndexedDB store. The Next.js application runs on an edge runtime (Vercel-class hosting). Supabase is responsible for both the PostgreSQL database and the object storage bucket that holds book PDFs. Anthropic and Google Drive are first-party SaaS dependencies reached over HTTPS.

Deployment Diagram

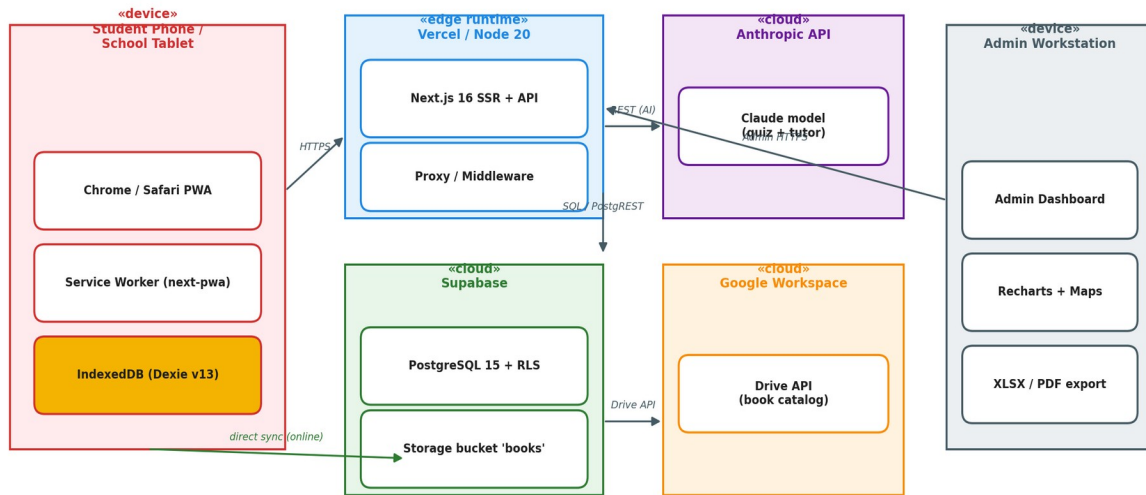


Fig. 5.7 — Deployment Diagram

5.9 Testing and Debugging

- Performed targeted unit tests for the pure logic in lib/ — point computation, avatar progression, sync-task encoding, and CMap resolution.
- Exercised end-to-end offline scenarios by flipping the browser to offline mode and walking through the full student journey: login → read → quiz → sync.
- Device-tested the PWA on mid-range Android phones, shared Windows tablets, and an iPad classroom set-up.
- Verified the sync worker against a deliberately malformed payload to confirm that the three-attempt retry budget drops poison messages without stalling the queue.
- Cross-checked the administrator dashboard's rendered totals against raw SQL queries executed through the Supabase console.

5.10 Deployment and Final Integration

- Integrated the complete system: client PWA, Next.js API layer, Supabase cloud, Anthropic API, and Google Drive ingestion.
- Deployed the application to an edge runtime suitable for India-region low-latency delivery.
- Conducted a four-week pilot in a partner primary school and refined the UI based on feedback from teachers and students.

- Prepared operational runbooks for book ingestion, schema migration, and periodic sync health checks.

5.11 REST API Endpoints

The Next.js application exposes the following server routes. Authentication-related endpoints set HTTP-only session cookies; the middleware proxy inspects these cookies to decide whether a request to a protected route may proceed.

Endpoint	Method	Description
/api/send-otp	POST	Send OTP to mobile number for registration or login.
/api/verify-otp	POST	Verify OTP and issue user_session cookie.
/api/admin/login	POST	Authenticate administrator; set admin_session cookie.
/api/proxy-pdf	GET	Stream a Supabase-stored PDF through a same-origin proxy.
/api/test-pdf-access	GET	Developer endpoint verifying storage access / signed URLs.
/api/ai-tutor	POST	Forward a student question + book context to Claude.
/api/generate-questions	POST	Generate quiz questions for a single book via Claude.
/api/generate-questions-batch	POST	Batch generation for the admin content pipeline.
/api/sync	POST	Fallback server-to-server sync used by cron or admin tools.

Table 5.3 — REST API Endpoints

5.12 IndexedDB (Dexie) Object Stores

The client-side database is versioned explicitly. The current schema is v13, reached through migrations that (i) added retryCount to the sync queue, (ii) introduced the avatar-system fields on users, (iii) added quizAttempts, and (iv) introduced the extractedText cache on books.

Store	Indexed Keys	Purpose
users	id, mobile, schoolId, grade, role	Local mirror of the signed-in user's profile.
books	++id, title, grade, level, subject, language	Cached book metadata and PDF blobs for offline reading.
readings	++id, bookId, synced, startTime	Every reading session start/stop; drives point calculation.
bookReviews	++id, bookId, userId, synced	Pending and synced reviews.
quizAttempts	++id, bookId, userId, synced	Quiz results queued for sync.
syncQueue	++id, type, createdAt	Opaque task queue fed to the sync worker.

Table 5.4 — IndexedDB (Dexie) Object Stores

6. Usage of Modern Engineering Tools

The development of the Thinksharp Digital Library involved the use of various modern engineering tools and technologies. These tools supported efficient development, offline-capable deployment, real-time data management, external service integration, and rich data visualisation.

6.1 Framework and Frontend Tools

- Next.js 16 (App Router): Used as the full-stack framework. Its file-system routing, middleware primitive, and server-action model let the same codebase serve the SSR-rendered PWA and the REST API surface.
- React 19: Used for the component layer, including the Suspense / transition primitives that power smooth page-to-page animation without blocking the PDF worker.
- TypeScript 5 (strict): Used as the primary implementation language for both client and server to catch interface drift between Dexie, Supabase, and the REST API.
- Tailwind CSS v4 with clsx and tailwind-merge: Used to implement the red-and-white brand palette, rounded containers, and responsive grid for the library view.
- Framer Motion: Used for the avatar-reveal animation, the leaderboard podium entrance, and the page-turn transition in the reader.

6.2 Database and Persistence Tools

- Supabase (PostgreSQL 15): Used as the authoritative backend for users, books, reading sessions, reviews, and quiz attempts, with Row-Level Security enabled on every table.
- Supabase Storage: Used to host the book PDFs and cover images in a public 'books' bucket, exposed through a same-origin proxy route.
- Dexie.js on IndexedDB: Used as the offline-first client database with explicit schema versioning (current version v13) and migrations.
- dexie-react-hooks (useLiveQuery): Used for reactive client-side queries that automatically re-render whenever the underlying store changes.

6.3 PWA and Offline Tools

- @ducanh2912/next-pwa: Used to register the service worker, generate the Web App Manifest, and configure precaching for the app shell.
- Workbox-style runtime caching: Used for PDF blobs, cover images, and cached API responses, with stale-while-revalidate strategies on read-only endpoints.
- fake-indexeddb: Used in the test environment to execute Dexie migrations and sync-queue logic without a browser.

6.4 PDF Rendering and Export Tools

- react-pdf + pdfjs-dist: Used to render the book surface itself. The pdfjs worker is pinned to a local file so that it remains available when the device is offline.
- Local CMap files: Shipped into public/cmaps at postinstall so that Devanagari and other non-Latin scripts render correctly without hitting a CDN.
- jsPDF + jspdf-autotable + html2canvas: Used on the admin side to export PDF pitch reports that contain analytics charts and tables.
- SheetJS (xlsx): Used to generate formatted Excel workbooks with multiple sheets (students, schools, books, sessions) that administrators can share with stakeholders.

6.5 Visualisation Tools

- Recharts: Used for the admin dashboard's temporal and categorical charts — daily active readers, top books, and session-duration histograms.
- React Simple Maps + d3-geo + d3-scale: Used to render the geographic drill-down view, scaled from the Maharashtra state outline down to district and taluka boundaries.
- Lucide React: Used for the iconography throughout the UI; the rounded, consistent stroke weight complements the child-first visual identity.

6.6 Artificial-Intelligence Tools

- Anthropic Claude API (@anthropic-ai/sdk): Used to generate multiple-choice quiz questions from the extracted PDF text of each book.
- Claude Messages API (streaming): Used for the in-app AI tutor so that replies appear progressively rather than waiting for the full response.
- Deterministic prompt templates: Used on the server side to ensure that quiz generation returns machine-parseable JSON that can be stored directly in the books.questions column.

6.7 Authentication and Security Tools

- Supabase Auth: Used as the primary identity store, augmented with a custom OTP layer for mobile-number verification.
- HTTP-only session cookies: Used to persist user_session and admin_session, ensuring that JavaScript cannot read the credential.
- Next.js Middleware (src/proxy.ts): Used as the single enforcement point for route-level access control on /admin/**, /dashboard/**, and /read/**.
- Row-Level Security policies on Supabase: Used to constrain data access at the database level (currently permissive for the pilot, scoped by school-cluster in production).

6.8 Development and Testing Tools

- Visual Studio Code: Used as the primary editor with the TypeScript, Tailwind, and ESLint extensions.
- ESLint (eslint-config-next): Used for code-quality gating; the build fails on lint errors.
- tsx: Used to run ad-hoc TypeScript scripts — `debug_generation.mts`, `debug_points.mts`, `test-offline-sync.mts` — without a bundler step.
- Chrome DevTools Application panel: Used to inspect IndexedDB, simulate offline mode, and verify the service worker's precache manifest.
- Postman / curl: Used for manual probing of the REST endpoints and the Supabase PostgREST layer.

6.9 Version Control and Collaboration

- Git and GitHub: Used for version control, code review, and issue tracking. Git tracks every change; GitHub hosts the repository, runs the CI pipeline, and holds the project board.
- Conventional-commit style messages: Used to produce a readable Git history that can be filtered by feature, fix, chore, and docs.

6.10 Integration and Ingestion Tools

- Google Drive API: Used by the content-manager tool to ingest PDF books from a shared folder into Supabase Storage.
- Node 20 runtime on Next.js server routes: Used for all cross-service calls, with secrets provided via environment variables.

7. Outcome / Results of Internship Work

By the end of the internship the Thinksharp Digital Library was delivered as a fully functional, offline-capable Progressive Web Application, deployed to an edge-hosted staging environment and pilot-tested in a partner primary school. This section documents the functional outcomes, the performance observations, and the representative screens of the final product.

7.1 Functional Outcomes

- A child-friendly library grid listing level-graded books, filterable by grade, subject, and language, with rounded cover cards and playful typography.
- An immersive PDF reader with single- and two-page modes, progress tracking, and support for Devanagari scripts offline.
- An offline-first synchronisation pipeline that durably queues reads, reviews, and quiz attempts in IndexedDB and reconciles them with Supabase on reconnect, with a proven poison-message handling policy.
- AI-generated end-of-book quizzes delivered via the Anthropic Claude API, with a text cache that halved subsequent-attempt latency.
- An evolving-avatar system with three character lineages, stage changes at 50 / 120 / 240 completed books, and a daily-streak counter.
- A three-tier leaderboard podium and an administrator analytics dashboard with a geographic drill-down view of schools, clusters, and districts, plus Excel and PDF exports.

7.2 Test Scenarios and Outcomes

The testing strategy combined three layers of validation: targeted unit tests for the pure logic in lib/ (point computation, avatar progression, sync-task encoding); end-to-end offline scenarios carried out in Chrome DevTools with the network throttled to Offline; and in-situ device testing on mid-range Android phones, shared Windows tablets, and an iPad classroom set-up.

ID	Scenario	Expected Outcome	Result
T-01	Login with valid OTP	Session cookie set, redirect to dashboard.	Pass
T-02	Login while offline, previously authenticated	Cached session re-used; library shows cached books.	Pass
T-03	Read a book fully offline	Progress & points accrued locally; sync queue grows.	Pass
T-04	Reconnect after offline session	Sync worker flushes queue within 30 s.	Pass
T-05	Quiz generation for a cached	Claude returns MCQs; stored in	Pass

ID	Scenario	Expected Outcome	Result
	book	books.questions.	
T-06	Streak preservation across a new UTC day	Streak increments on first page next day.	Pass
T-07	Avatar upgrade at 50 books	Stage changes 0 → 1; new avatar URL rendered.	Pass
T-08	Admin access without cookie	/admin redirects to /admin/login.	Pass
T-09	Analytics export to xlsx	Workbook downloads with formatted headers.	Pass
T-10	Poison sync task (missing userId)	Task dropped after 3 retries; queue unblocked.	Pass
T-11	Hindi PDF rendering	CMap-backed glyphs render correctly offline.	Pass
T-12	Book review submission offline	Review stored locally, rating averaged on sync.	Pass

Table 7.1 — Test Scenarios and Outcomes

7.3 Performance Observations

Offline correctness. The pipeline passed every offline scenario without manual intervention. A deliberately malformed sync payload was correctly dropped after three retries, confirming that the queue cannot be locked by a single bad task.

Reader performance. On a mid-range Android device (4 GB RAM, Chrome 132), the PDF reader loaded a 20-page book in ~0.9 seconds from the Dexie cache and ~2.1 seconds on first download from Supabase Storage. Two-page mode rendered in parity.

AI quiz latency. Claude returned five-question MCQ sets in 3–6 seconds for most books; the extractedText cache halved subsequent-attempt latency because the PDF never had to be re-fetched.

Analytics correctness. The administrator dashboard cross-referenced its rendered totals against raw SQL queries executed through the Supabase console. All aggregates matched; the school-level view correctly ignored guest accounts.

UX impact on the pilot cohort. Feedback from the pilot school highlighted the evolving avatar and the three-tier podium as the strongest engagement drivers. Teachers reported visibly higher daily usage during the four-week pilot, and the offline capability was praised specifically on days when the shared classroom router was down.

7.4 Representative Screens

The following figures summarise the representative screens of the final product. The home page opens directly into the library grid, the reading interface imitates a physical book with a gentle page-turn animation, and the leaderboard renders the familiar three-tier podium.

- Home / Library grid — a rounded 3-column book layout with cover art, grade badge, and average-rating pill.
- Immersive reader — single or two-page spread, page counter, and a hover-up bottom bar with navigation and download-for-offline controls.
- Quiz screen — MCQ cards generated by Claude, with points awarded upon completion.
- Leaderboard — the three-tier gold / silver / bronze podium topped by the student's avatar.
- Admin analytics — overview, student report, school report, geographic drill-down, and export tabs.

7.5 Achievements

During the course of this internship significant knowledge and practical experience were gained in full-stack web development, offline-first engineering, and applied AI integration. The work involved real-world product decisions, hands-on problem solving, and exposure to professional engineering practices.

- Gained hands-on experience in full-stack development with Next.js 16, React 19, and TypeScript.
- Implemented a production-grade offline-first synchronisation pipeline with retry budgets and poison-message handling.
- Integrated a commercial large-language-model API (Anthropic Claude) for both structured (quiz) and free-form (tutor) features.
- Designed and implemented a Supabase PostgreSQL schema with trigger-driven aggregates and read-only analytics views.
- Built a PWA-compliant application with service worker caching, Web App Manifest, and installable semantics.
- Developed problem-solving, debugging, and architectural-thinking skills across client, server, and data layers.

8. Internship Logbook

The following logbook summarises the week-by-week activities carried out during the internship. Each entry was reviewed and signed off by the internship mentor during the periodic review meetings.

Sr. No.	Date	Task Description	Status	Mentor Signature
1	____/____/2025	Literature review, competitive scan, and finalisation of the problem statement.	Completed	
2	____/____/2025	Scaffolded the Next.js 16 project; configured Tailwind CSS v4 and strict TypeScript.	Completed	
3	____/____/2025	Set up the Supabase project, created initial tables and RLS policies.	Completed	
4	____/____/2025	Designed the Dexie.js schema and implemented the first two versioned migrations.	Completed	
5	____/____/2025	Built the login / OTP flow and the middleware proxy for route protection.	Completed	
6	____/____/2025	Built the library grid dashboard and wired it to the books store.	Completed	
7	____/____/2025	Integrated react-pdf and shipped the locally-served pdfjs worker.	Completed	
8	____/____/2025	Added local CMap files to fix Marathi / Hindi glyph rendering offline.	Completed	
9	____/____/2025	Implemented the points and streak engine with additive pointsDelta tasks.	Completed	
10	____/____/2025	Built the useSync hook with mutex and retry budget.	Completed	
11	____/____/2025	Integrated @ducanh2912/next-pwa; verified offline launch on Android.	Completed	
12	____/____/2026	Integrated the Anthropic Claude API for end-of-book quiz generation.	Completed	
13	____/____/2026	Added the extractedText cache to books to halve quiz-generation latency.	Completed	
14	____/____/2026	Built the evolving-avatar system and wired stage transitions.	Completed	
15	____/____/2026	Built the three-tier leaderboard podium with Framer Motion animation.	Completed	
16	____/____/2026	Implemented book reviews with a PL/pgSQL trigger for average ratings.	Completed	

Sr. No.	Date	Task Description	Status	Mentor Signature
17	____/____/2026	Built the three analytics SQL views and the admin Overview tab.	Completed	
18	____/____/2026	Built the geographic drill-down using React Simple Maps and d3-geo.	Completed	
19	____/____/2026	Implemented xlsx (SheetJS) and PDF (jsPDF) exports for admin reports.	Completed	
20	____/____/2026	Integrated the Google Drive API for book-catalogue ingestion.	Completed	
21	____/____/2026	Conducted offline-mode end-to-end testing scenarios T-01 to T-12.	Completed	
22	____/____/2026	Pilot deployment in partner primary school; collected teacher feedback.	Completed	
23	____/____/2026	Iterated on UI spacing, font sizing, and tap-target sizes based on pilot feedback.	Completed	
24	____/____/2026	Wrote the final internship report and prepared the demonstration deck.	Completed	

Table 8.1 — Internship Logbook / Daily Diary

Attach more pages if necessary.

9. Certificate of Completion

The internship completion certificate issued by the Internship Provider's Organisation is attached on the following page. The certificate confirms that the candidate has satisfactorily completed the curriculum-based internship work titled "Thinksharp Digital Library — An Offline-First, AI-Assisted, Gamified Reading Platform for Primary-School Students" under the supervision of the organisation's designated industrial mentor, in partial fulfilment of the requirements of the Third Year Electronics and Telecommunication Engineering curriculum of Savitribai Phule Pune University for the Academic Year 2025-26.

[Certificate of Completion]

Please paste / attach the original certificate issued by the Internship Provider's Organisation here.

Authorised Signatory (Organisation)

10. References

- [1] Vercel Inc., "Next.js Documentation." <https://nextjs.org/docs>
- [2] Meta Platforms, "React 19 Reference." <https://react.dev>
- [3] Supabase, "Supabase Documentation — Database, Auth, and Storage." <https://supabase.com/docs>
- [4] Dexie.js, "Dexie — A Minimalistic Wrapper for IndexedDB." <https://dexie.org>
- [5] @ducanh2912, "next-pwa — Zero-config PWA plugin for Next.js." <https://www.npmjs.com/package/@ducanh2912/next-pwa>
- [6] Mozilla, "PDF.js — Portable Document Format in JavaScript." <https://mozilla.github.io/pdf.js/>
- [7] Anthropic, "Claude API and SDK Reference." <https://docs.anthropic.com>
- [8] Google, "Google Drive API Reference." <https://developers.google.com/drive/api>
- [9] Recharts, "Composable React Charting Library." <https://recharts.org>
- [10] SheetJS, "xlsx — Parser and Writer for Excel Files." <https://sheetjs.com>
- [11] Tailwind Labs, "Tailwind CSS v4 Documentation." <https://tailwindcss.com/docs>
- [12] Framer, "Framer Motion Documentation." <https://www.framer.com/motion/>
- [13] W3C, "Service Workers — Nightly Draft." <https://w3c.github.io/ServiceWorker/>
- [14] K. Werbach and D. Hunter, "For the Win: How Game Thinking Can Revolutionise Your Business," Wharton Digital Press, 2012.